

Design and Implementation of a Reliability-Aware Study Group Matcher Web Application

Victor Googe

Department of Computer Science

Georgia State University

Email: vgooge1@student.gsu.edu

Abstract—This paper presents the design and full-stack implementation of a web application for matching college students with study groups based on course enrollment, schedule availability, and study preferences. The project extends a conventional study-group platform by introducing a reliability and reputation system that promotes dependable participation. The application was built for CSC 4370 using Vue 3, Vite, Vue Router, Pinia, Node.js, Express, and SQLite. The completed prototype delivers a working client-server architecture with JWT-based authentication, a relational data model covering users, groups, sessions, attendance, and ratings, and an explainable scoring formula that combines attendance history, peer feedback, and no-show penalties. The system supports the full lifecycle of a study group: profile creation, availability tracking, group browsing with match scoring, join-request approval, session scheduling, live attendance check-in, post-session peer rating, and leader-initiated student invitations. The project demonstrates that a focused, reliability-aware feature set can be designed, implemented, and demonstrated within a single semester using modern JavaScript tools.

Index Terms—Vue.js, Node.js, Express, SQLite, Web Programming, Study Groups, Reliability System, Full-Stack Development, JWT Authentication

I. INTRODUCTION

Many students benefit from studying in groups, but finding a group that is both relevant and dependable can be difficult. Existing coordination methods, such as informal group chats or class discussion boards, do not help students evaluate whether other members are likely to show up, contribute, and remain engaged over time. Students may join groups that appear promising but fail due to repeated absences or poor follow-through.

The goal of this project was to design and build a *Study Group Matcher* web application that helps students create and join study groups while also providing a reliability-aware view of membership quality. Instead of matching students only by course or schedule, the system introduces a computed reliability score based on attendance records and peer ratings. This score is displayed throughout the application so that students and group leaders can make more informed decisions when forming or joining groups.

The main contributions of this project are as follows:

- A fully working full-stack prototype aligned with the tools and concepts covered in the course.

- A relational database schema modeling users, profiles, study groups, sessions, attendance, ratings, invitations, and reliability snapshots.
- A transparent reliability-scoring formula that rewards consistent attendance and penalizes no-shows.
- A complete user workflow from account registration through group membership, session attendance, and peer rating.
- A contextual group leadership model that allows any student to create and lead groups without requiring a separate privileged account role.

II. PROJECT OVERVIEW AND REQUIREMENTS

The project was based on *Theme D — Study Group Matcher*, specifically *Variant D4 — Rating / Reliability System*. The system was designed around two account-level roles and one contextual group role:

- **Student:** create a profile, set availability, browse groups, request to join groups, attend sessions, self-check-in during live sessions, and rate peers after completion.
- **Teacher:** same capabilities as a Student plus elevated platform-oversight permissions, equivalent to an administrator.
- **Group Leader (contextual):** any student who creates a group becomes its leader. The leader approves or rejects join requests, schedules sessions, starts and completes sessions, records attendance, and invites matched students. This is a per-group role that does not require a special account type.

The key project requirement was that the application support normal study-group workflows and include a mechanism for representing how dependable a member has been in prior group participation. This requirement informed every major design decision, from the database schema to the UI layout.

III. SYSTEM ARCHITECTURE

The application follows a standard client-server architecture with a REST API boundary separating the frontend from the backend.

A. Frontend

The frontend is built with Vue 3 and Vite. Vue Router manages client-side navigation, and Pinia manages authentication state globally. Heroicons provide consistent iconography throughout the interface. The major views include:

- Home (landing page with call-to-action),
- Login and Register pages,
- Dashboard (reliability summary, upcoming sessions, group panel, onboarding progress),
- Profile (edit academic info, study style, and recurring availability blocks),
- Groups browse page (search, format filter, match scoring, and fit labels),
- Group detail page (leader layout and member layout variants),
- My Groups (groups led and groups joined, with member count and location),
- Session detail page (attendance management, self check-in, and peer rating).

B. Backend

The backend uses Node.js and Express. Route modules are separated by domain: authentication, users, groups, sessions, ratings, and invitations. JWT-based middleware protects all authenticated routes. A `requireAuth` middleware validates the bearer token on every protected request, while a `requireRole` utility can restrict specific actions to Teacher-level accounts.

C. Database Layer

SQLite was selected as the persistence layer because it requires no server process, is easy to seed and reset during development, and still supports meaningful relational modeling with foreign keys, constraints, and composite unique indexes.

IV. DATABASE DESIGN

The schema was designed around the real entities involved in study-group coordination. Table I summarizes the core tables.

The schema enforces data integrity through foreign-key constraints, `UNIQUE` indexes on composite keys such as (`session_id`, `user_id`) in the attendance table, and `CHECK` constraints on enumerated columns such as session status and attendance status.

V. RELIABILITY AND MATCHING DESIGN

The most distinctive feature of the application is its reliability-aware matching model. Rather than a black-box algorithm, the score was intentionally designed to be transparent and explainable. Three inputs are combined:

- attendance rate A (sessions attended divided by sessions as an active member),
- average peer rating R (mean of all received ratings on a 1–5 scale),
- no-show penalty P (a fixed deduction per confirmed no-show record).

The scoring formula is shown in (1):

$$\text{Score} = \text{clamp}(\text{round}(A \times 70 + \frac{R}{5} \times 30 - P), 0, 100) \quad (1)$$

The score is mapped to four human-readable tiers, each displayed with a contextual icon throughout the UI:

TABLE I
CORE DATABASE TABLES

Table	Purpose
users	Login identity, role (student or teacher), and timestamps
profiles	Academic major, bio, study style, course list, group-size preference, and looking-for-group flag
availability_blocks	Recurring day-of-week windows used for schedule matching
study_groups	Title, course code, description, format, location, meeting link, capacity, and leader reference
group_members	Accepted memberships with active/removed status
group_join_requests	Pending requests awaiting leader decision
group_invitations	Leader-initiated invitations sent to matched students
sessions	Scheduled meetings with status lifecycle: scheduled, active, completed, cancelled, missed, rescheduled
attendance	Per-session records: present, absent, excused, or no-show
ratings	Post-session peer ratings (1–5 scale) with optional text and a flagged field
reliability_snapshots	Cached computed reliability scores for efficient retrieval

- **Highly Reliable** (≥ 90) — verified badge icon,
- **Reliable** (≥ 75) — thumbs-up icon,
- **Mixed** (≥ 50) — exclamation-circle icon,
- **Low Reliability** (< 50) — warning triangle icon,
- **New** (no history) — sparkle icon.

Group browse results are sorted by a computed match score that incorporates course overlap, availability overlap, study-style alignment, and the viewer’s own reliability tier. Match reasons are surfaced on each group card so that students can evaluate fit at a glance before requesting to join.

Abuse protections built into the rating system include:

- users cannot rate themselves,
- ratings are restricted to members who attended the same session (present status required),
- a unique constraint prevents duplicate rater–ratee–session combinations,
- a flagged field marks suspicious ratings for future moderation.

VI. API DESIGN

The backend exposes REST-style endpoints organized by resource domain. Table II lists the endpoints implemented in the prototype.

VII. APPLICATION WORKFLOW

A complete end-to-end workflow in the implemented system proceeds as follows:

- 1) A student registers an account, selecting a role of Student or Teacher.
- 2) The student completes a profile: major, bio, study style, course list, and recurring availability blocks.

TABLE II
API ENDPOINTS

Method	Endpoint
POST	/api/auth/register
POST	/api/auth/login
GET	/api/auth/me
GET	/api/users/me
PUT	/api/users/me
POST	/api/users/me/availability
DELETE	/api/users/me/availability/:id
GET	/api/users/:id/reliability
GET	/api/groups
POST	/api/groups
GET	/api/groups/mine
GET	/api/groups/:id
PUT	/api/groups/:id
DELETE	/api/groups/:id
POST	/api/groups/:id/join-request
POST	/api/groups/:id/requests/:id/approve
POST	/api/groups/:id/requests/:id/reject
DELETE	/api/groups/:id/members/:id
GET	/api/groups/:id/student-matches
POST	/api/groups/:id/invitations
GET	/api/sessions/upcoming
GET	/api/groups/:id/sessions
POST	/api/groups/:id/sessions
GET	/api/sessions/:id
PATCH	/api/sessions/:id/start
PATCH	/api/sessions/:id/complete
PATCH	/api/sessions/:id/cancel
PATCH	/api/sessions/:id/reschedule
POST	/api/sessions/:id/check-in
POST	/api/sessions/:id/attendance
POST	/api/sessions/:id/ratings
GET	/api/invitations
PATCH	/api/invitations/:id/respond

- 3) The student browses the group listing, which is sorted by computed match score and annotated with fit labels and match reasons.
- 4) The student submits a join request to a group. The group leader sees the request in the pending-requests panel on the group detail page.
- 5) The leader approves or rejects the request. Approved members gain access to the group’s meeting link.
- 6) The leader schedules a session with a title, date, duration, location, and optional preparation notes.
- 7) When the session time arrives, the leader starts it, transitioning its status to active.
- 8) Active members navigate to the session detail page and self-check-in, recording a present attendance entry.
- 9) The leader completes the session, which automatically marks any member who did not check in as absent.
- 10) Members who were marked present can now rate one another on a 1–5 scale with optional written feedback.
- 11) Reliability scores are recomputed from the updated attendance and ratings and are reflected immediately in badges throughout the application.
- 12) Leaders can separately use the *Find & Invite Students* panel to surface matched students and send direct invitations, which recipients can accept or decline from their dashboard.

VIII. IMPLEMENTATION DETAILS

A. Frontend

The frontend is organized into page-level views and shared components. Key components include:

- `AppHeader` — navigation bar with a role-colored user pill (green for Student, orange for Teacher) and active-route highlighting.
- `ReliabilityBadge` — renders a tier label, numeric score, and contextual icon; used consistently across group cards, member lists, and match results.
- `GroupCard` — summarizes a group with a fit label, match reasons, map-pin location, and member count; groups the user has already joined are visually de-emphasized.

The dashboard adapts to user state: new users see a step-by-step onboarding checklist that auto-hides after completing all steps and viewing the completion state three times. The dashboard also displays an upcoming sessions panel sorted by scheduled date and a live-session alert banner when any of the user’s groups has an active session.

Group detail pages render two distinct layouts depending on the viewer’s relationship to the group. The leader layout places the session scheduling form on the left and stacks members, invite, sessions, and pending requests on the right. The member layout places the sessions list on the left and the members list on the right, with no scheduling form or invite panel shown.

B. Backend

Session status transitions are enforced server-side. A session can only be started if its status is `scheduled` or `missed`, and it can only be completed if it is `active`. A reconciliation function runs on every group session list request and automatically promotes stale scheduled sessions to `missed` when their scheduled time has passed, ensuring the session history is always accurate without requiring a background process.

The `/api/groups/mine` route joins the user’s led and joined groups in a single query and annotates each row with an `isLeader` boolean, allowing the frontend to render the correct layout and labels without a follow-up request.

C. Reliability Computation

The reliability service computes scores on demand from live attendance and rating data:

Listing 1. Core reliability scoring logic

```
function calculateReliabilityScore(
  attendanceRate, avgRating, noShows) {
  const noShowPenalty = noShows * 10;
  const score = Math.round(
    (attendanceRate * 70) +
    ((avgRating / 5) * 30) -
    noShowPenalty
  );
  return Math.max(0, Math.min(100, score));
}
```

Attendance carries 70% of the weight because it is the primary observable signal of dependability. Peer ratings contribute the remaining 30% as a qualitative component. Confirmed no-shows carry an explicit 10-point penalty each, so a pattern of absences can overcome an otherwise favorable rating average.

IX. OUTCOMES

The project delivered a complete, demonstration-ready full-stack prototype. The main outcomes include:

- A working JWT authentication and authorization system with role-aware access control.
- A complete group management workflow: create, browse with match scoring, request, approve, and invite.
- A full session lifecycle: schedule, start, self-check-in, complete, cancel, and reschedule.
- A peer rating system constrained by session participation with a live-computed reliability score.
- A consistent, icon-rich UI that reflects reliability tiers, group membership status, and session states throughout every view.
- A contextual leadership model that removes the dedicated leader account role, making the system more flexible for real student use.

X. TESTING AND EVALUATION

Evaluation focused on scenario-based functional testing of core user flows and data constraint enforcement. The prototype was seeded with representative data including multiple users, groups of different formats, past completed sessions, and existing attendance and rating records to simulate a realistic demo environment.

TABLE III
FUNCTIONAL TESTING SCENARIOS

Scenario	Status
User registration, login, and JWT issuance	Passed
Profile editing and availability management	Passed
Group creation with location and meeting link	Passed
Group browse, search, and match-score sorting	Passed
Join request submission and leader approval	Passed
Session scheduling, start, check-in, complete	Passed
Automatic absent marking on session completion	Passed
Peer rating with participation constraint	Passed
Reliability badge rendering by score tier	Passed
Student invitation via match results	Passed
Leader vs. member layout on group detail page	Passed
Role-based UI (Student green, Teacher orange)	Passed

XI. CHALLENGES

A. Balancing Scope and Feasibility

A major challenge was keeping the project within a realistic semester scope. Matching logic, attendance tracking, role management, and rating moderation can each grow into substantial sub-projects. The system was therefore built in layers, prioritizing a working MVP before adding secondary features and UI polish.

B. Contextual vs. Account-Level Leadership

An early design question was whether group leadership should be a permanent account role or a per-group property. The final design removes the dedicated `leader` account role and makes any student who creates a group its leader automatically. This eliminated a category of access-control complexity and better reflects real-world behavior, where a student may lead one group while being a member in another.

C. Designing a Fair Reputation System

Designing a reliability model that is useful without being opaque required careful weighting. A score that is too simple may be inaccurate; one that is too complex becomes difficult to explain and justify. The final formula is transparent, weights attendance as the primary signal, and is constrained by participation rules that reduce the risk of score manipulation.

D. Preventing Rating Abuse

Any peer-rating feature introduces risks of spam, retaliation, or score inflation through collusion. The design addresses this through session-based eligibility checks, unique constraints on rater–ratee–session triples, and a `flagged` column available for future moderation workflows.

XII. FUTURE WORK

While the core prototype is complete, several extensions would improve the application for a production deployment:

- email notifications for join-request decisions, session reminders, and invitation responses,
- dispute and moderation workflows for flagged ratings,
- reliability trend charts and attendance history visualizations on the profile page,
- fully mobile-optimized responsive layouts,
- server-side pagination for large group and session lists,
- automated integration and end-to-end test suites.

XIII. LLM USAGE

Large language model tools were used as development assistants throughout planning, implementation, and documentation. They were helpful for brainstorming architecture, generating boilerplate, debugging logic, and organizing code structure. All project decisions, scope trade-offs, feature prioritization, and correctness verification required manual reasoning and testing. LLMs accelerated development support tasks rather than replacing the need to understand the application.

XIV. COURSE FEEDBACK

This course was effective in connecting foundational web concepts with a larger final project. The emphasis on modern tools — Vue, Vite, Express, GitHub, and SQLite-backed application design — produced a practical and portfolio-ready result. One improvement for future offerings would be an earlier intermediate checkpoint focused specifically on deployment readiness and full end-to-end integration. This would help students identify integration risks sooner and arrive at the final demo with less last-minute pressure.

XV. CONCLUSION

This project demonstrates the design and complete prototype implementation of a reliability-aware study group matcher web application for college students. The system combines conventional study-group management features with a distinct reputation component that improves the quality of group matching. The finished prototype covers the full lifecycle from registration through peer rating, uses a clean role model that separates account-level identity from group-level leadership, and presents reliability information consistently and legibly throughout the interface. The work confirms that a focused, well-scoped feature set can be fully implemented within a single semester using modern JavaScript tools and provides a solid foundation for a more complete production application in future iterations.

REFERENCES

- [1] Vue.js Documentation. Available: <https://vuejs.org>
- [2] Vite Documentation. Available: <https://vitejs.dev>
- [3] Node.js Documentation. Available: <https://nodejs.org>
- [4] Express Documentation. Available: <https://expressjs.com>
- [5] SQLite Documentation. Available: <https://sqlite.org>
- [6] Pinia State Management. Available: <https://pinia.vuejs.org>
- [7] Vue Router Documentation. Available: <https://router.vuejs.org>
- [8] Heroicons. Available: <https://heroicons.com>
- [9] JSON Web Tokens. Available: <https://jwt.io>